TP5 : Algorithme de Shor

Avant-propos

Bienvenue au TP5. Si les TPs précédents jouaient encore dans le bac à sable des algorithmes-jouets, celui-ci joue dans la cour des grands : vous allez implémenter l'algorithme qui a, en 1994, transformé l'informatique quantique d'une curiosité théorique en menace existentielle pour la cryptographie moderne. Peter Shor a montré qu'un ordinateur quantique pouvait factoriser des entiers en temps polynomial, là où le meilleur algorithme classique connu (le crible algébrique) reste sous-exponentiel. Comme la sécurité de RSA repose précisément sur la difficulté supposée de la factorisation, l'algorithme de Shor est la raison pour laquelle le NIST a passé cinq ans à standardiser des protocoles post-quantiques. Vous vous apprêtez donc à coder, pour la note, l'objet qui rend obsolète une bonne partie d'internet.

Quelques bonnes nouvelles avant de paniquer :

— RSA-2048 ne tombera pas grâce à votre rendu. Le simulateur quantique gérera péniblement la factorisation d'un nombre à 5 chiffres. Pour casser RSA, il faudrait environ 20 millions de qubits physiques tolérants aux fautes ; la communauté en est à quelques centaines.

— L'algorithme est en réalité *mostly classique*. Le cœur quantique fait une seule chose : trouver la *période* d'une fonction modulaire. Tout le reste, c'est de l'arithmétique modulaire classique et un peu d'algorithme d'Euclide.

— Vous avez déjà presque tout fait. Le cœur quantique est une variante du QPE du TP3, et le post-traitement classique est essentiellement de la théorie des nombres élémentaire.

Quelques conseils de survie :

— **L'arithmétique modulaire impose une discipline.** Chaque opération doit être réversible (donc adjointable), ce qui veut dire : pas de `Reset` sauvage. Si vous allouez un qubit ancillaire, vous le devez à zéro en sortie, sans tricher. C'est non négociable parce que toute la fonction s'utilise ensuite comme une porte contrôlée à l'intérieur de QPE.

— **Réutilisez le TP3.** L'estimation de phase ici est une variante itérative qui économise les ancillas, mais l'idée reste : on accumule une phase $e^{2\pi i\varphi}$, on lit φ avec la QFT inverse. Le détail de l'implémentation change, le principe non.

— **L'optimisation a^{2^p} n'est pas optionnelle.** L'algorithme naïf appelle U_a exactement 2^p fois ; pour $p = 17$, c'est 131 072 appels. Avec `ExpModI(a, 2^p, N)`, c'est un seul appel à $U_{a^{2^p} \bmod N}$. La différence entre "ça termine en 5 secondes" et "vous le lancez avant le repas et vous mangez froid".

— **La construction `repeat-until-fixup` est votre amie**. Shor est probabiliste de bout en bout : la mesure quantique peut rater, le générateur tiré au hasard peut être de mauvaise foi, la période peut être impaire. Le `repeat` avec `fixup` permet de retomber sur ses pieds sans pleurer.

— **N'écrivez pas votre propre algorithme d'Euclide.** `GreatestCommonDivisorI`, `InverseModI`, `ContinuedFractionConvergentI`, `ExpModI`, `BitSizeI` existent dans la bibliothèque standard. Ce sont précisément les ponts entre le calcul classique et le quantique. Servez-vous.

Chaque exercice est accompagné d'un encadré **Indice** pour vous débloquer et d'un encadré **Test** avec des cas concrets à passer. L'annexe A explique *pourquoi* tout cela fonctionne, en particulier

la réduction non triviale entre factorisation et recherche de période. Si vous n'avez jamais vu pourquoi factoriser un entier revient à trouver un ordre dans un groupe multiplicatif, c'est la première chose à lire.

1 Partie quantique : la porte U

Ces exercices se trouvent dans le fichier `UGate.qs`.

Le cœur arithmétique de Shor est la porte U_a qui, sur un registre $|x\rangle$, calcule la multiplication modulaire :

$$U_a |x\rangle = |a \cdot x \bmod N\rangle$$

Pour la construire, on procède par étapes : addition modulaire d'une constante, puis multiplication modulaire d'une constante (qui se décompose en additions). Chaque opération doit être `Adj + Ctl` parce qu'elle sera appelée contrôlée à l'intérieur du QPE.

Vue d'ensemble. Dans cette partie, vous construisez progressivement la porte U_a telle que

$$U_a |x\rangle = |ax \bmod N\rangle$$

Cette porte sera ensuite utilisée dans l'algorithme d'estimation de phase (QPE) pour extraire la période r.

Pipeline global :

$$\text{arithmétique modulaire} \to U_a \to \text{QPE} \to \frac{s}{r}$$

1.1 Addition modulaire d'une constante

L'objectif est d'implémenter la porte

$$\text{AddConstantModulo}(c, N) |x\rangle = |(x + c) \bmod N\rangle$$

La signature attendue dans le squelette :

```
operation AddConstantModulo(c: Int, modulo: Int, qs: Qubit[]) : Unit is Adj + Ctl
```

Les variables d'entrée, en détail.

- `c: Int` est la constante *classique* à ajouter. On suppose $0 \leq c < N$; si $c \geq N$, faites-le passer modulo N en amont.
- `modulo: Int` est le module N, lui aussi classique.
- `qs: Qubit[]` est le registre quantique encodant x en little-endian (`qs[0]` est le bit de poids faible). On suppose $0 \leq x < N$ en entrée ; si $x \geq N$, le comportement n'est pas défini (et personne ne sera évalué là-dessus, mais ne plantez pas vos propres tests pour autant).
- Le suffixe `is Adj + Ctl` *n'est pas optionnel* : la porte sera utilisée, comme on va le voir, comme `Adjoint AddConstantModulo` dans la décomputation des ancillas de la fonction `MultiplyConstantModulo`, et comme `Controlled AddConstantModulo` à plein d'endroits. Si vous écrivez quelque chose qui n'admet pas l'adjoint (`Reset`, mesures intermédiaires...), le compilateur vous le dira au moment de l'instanciation.

Stratégie générale. On veut $x + c \bmod N$, mais on ne dispose que d'un additionneur entier non modulaire (`IncByI` de la bibliothèque standard). L'astuce classique consiste à : ajouter c, soustraire N pour "rectifier" si le résultat aurait dépassé $N - 1$, puis re-ajouter N uniquement si

la soustraction a sous-débordé. La détection du sous-débordement passe par un qubit de **retenue** (`carry`) qui mémorise le bit de signe. Le défi : remettre ce `carry` à zéro en sortie *sans utiliser `Reset`*.

L'enchaînement standard :

1. Allouer un qubit `carry` (initialement $|0\rangle$).
2. Ajouter c au registre étendu `qs + [carry]`, où le carry sert de bit de poids fort.
3. Soustraire N via `Adjoint IncByI(N, qs + [carry])`. Si l'on est passé en "négatif" au sens entier signé, `carry` vaut 1.
4. Conditionnellement à `carry`, ré-ajouter N avec `Controlled IncByI([carry], (N, qs))`.
5. Remettre `carry` à zéro en testant une condition sur le résultat (voir `ApplyIfLessOrEqualL`).

Pourquoi le carry ? Sans lui, on ne peut pas distinguer "$x + c$ a débordé" de "$x + c$ n'a pas débordé". L'astuce centrale est que la soustraction de N rend la première situation négative et la seconde positive ; le bit de signe est exactement l'information dont on a besoin.

Pourquoi ne peut-on pas faire un simple `Reset` sur le carry ? Parce que `Reset` mesure le qubit, et la mesure ne supporte pas l'opérateur `Adjoint`. Sans `Adj`, votre porte U ne s'inverse pas, et tout l'algorithme s'effondre (les boucles `within ... apply`, le QPE, etc. demandent tous l'adjoint).

Indice. `IncByI(c, qs + [carry])` ajoute c à l'entier représenté par la concaténation `qs ++ [carry]`, le `carry` servant de bit de poids fort. Pour la rectification finale du carry, observez que dans le cas "pas de débordement initial", le résultat final est $\geq c$, et dans le cas "débordement", il est $< c$. `ApplyIfLessOrEqualL(X, IntAsBigInt(c - 1), qs, carry)` ou son équivalent flippe alors le carry exactement quand il le faut (le choix exact de la condition dépend de la valeur du carry après l'étape 3, à vous de tracer).

Si vous bloquez, déroulez à la main sur un exemple : $c = 5$, $N = 7$, $x = 4$. Étape 2 : $4+5 = 9$, donc `qs+carry` contient 9 (`qs` = 1, `carry` = 1). Étape 3 : $9 - 7 = 2$, donc `qs+carry` = 2 (`qs` = 2, `carry` = 0). Étape 4 : `carry` = 0, donc rien. Étape 5 : il faut remettre `carry` à 0 (déjà fait). Résultat : qs = $2 = 4 + 5 \bmod 7$. ✓

Test. *Rappel : `DumpMachine` affiche les bases en little-endian, donc l'état $|$`100`$\rangle$ correspond à l'entier 1, pas à 4.*

Trois cas single-shot à vérifier avec `DumpMachine` après chaque appel :

- $x = 4$, $c = 2$, $N = 7$: préparez `X(qs[2])` (donne $|001\rangle$ en little-endian = 4), appliquez `AddConstantModulo(2, 7, qs)`. Attendu : $|$`011`$\rangle = 6$ (pas de débordement).
- $x = 6$, $c = 3$, $N = 7$: préparez `X(qs[1]); X(qs[2])` (donne $|011\rangle = 6$), appliquez `AddConstantModulo(3, 7, qs)`. Attendu : $|$`010`$\rangle = 2$ (débordement, $6+3 = 9 \bmod 7 = 2$).
- Cas résultat zéro : $x = 2$, $c = 5$, $N = 7$. Attendu : $|$`000`$\rangle = 0$.

Test du cycle complet (qui valide implicitement l'adjoint et la décomputation propre du `carry`) : préparez $|x\rangle = |1\rangle$ et appliquez `AddConstantModulo(1, 7, qs)` sept fois de suite. Vous devez retomber exactement sur $|1\rangle$, soit $|$`100`$\rangle$ en little-endian. Si le programme crashe avec "Qubit released while not in $|0\rangle$ state", votre `carry` est mal décomputé quelque part.

Checkpoint. Si vous êtes arrivé ici avec un code qui passe les tests, vous savez maintenant :

- implémenter une opération arithmétique **réversible** ;
- gérer un qubit ancillaire sans utiliser `Reset` ;
- détecter et corriger un débordement en arithmétique modulaire.

C'est la première brique fondamentale : **tout le reste repose dessus**. Si cette partie n'est pas parfaitement comprise, la suite deviendra rapidement opaque.

Questions (à réfléchir, non notées mais qui seront posé à l'oral).

— Que se passerait-il si on supprimait le qubit `carry` ?

— Pourquoi un `Reset` casserait-il la propriété `Adj` ?

1.2 Multiplication modulaire d'une constante

L'objectif est d'implémenter la porte

$$\text{MultiplyConstantModulo}(c, N)\,|x\rangle = |(c \cdot x) \bmod N\rangle$$

La signature attendue :

```
operation MultiplyConstantModulo(c: Int, modulo: Int, qs: Qubit[]) : Unit is Adj
+ Ctl
```

Les variables d'entrée, en détail.

— `c: Int` est la constante multiplicative *classique*. Préconditions importantes : $0 < c < N$ et $\gcd(c, N) = 1$. La première garantit qu'on reste dans $[0, N)$; la seconde est obligatoire parce que la décomputation utilise `InverseModI(c, N)`, qui n'existe que si c et N sont premiers entre eux.

— `modulo: Int` est le module N, classique.

— `qs: Qubit[]` encode x en little-endian. En sortie, `qs` contient $cx \bmod N$ (le registre est écrasé). On suppose $0 \leq x < N$ en entrée.

Stratégie. On ne peut pas multiplier "en place" : l'opération $|x\rangle \to |cx \bmod N\rangle$ détruit l'information sur l'ancien x, donc elle n'est pas réversible si elle modifie directement `qs`. On passe par un registre auxiliaire, en utilisant la **décomposition binaire** du produit :

$$c \cdot x = \sum_{i=0}^{n-1} x_i \cdot (c \cdot 2^i) \quad \text{donc} \quad c \cdot x \bmod N = \sum_{i=0}^{n-1} x_i \cdot (c \cdot 2^i \bmod N) \pmod N$$

où x_i est le i-ème bit de x. Chaque terme de la somme est soit "ajouter $c \cdot 2^i \bmod N$ à l'ancilla", soit "ne rien faire", selon que x_i vaut 1 ou 0. On a donc n additions modulaires contrôlées par les bits de `qs`.

L'enchaînement :

1. Allouer un registre ancillaire `aux` de même taille que `qs`, initialement $|0\rangle$.
2. Pour chaque $i \in [0, n-1]$: `Controlled AddConstantModulo([qs[i]], (c · 2^i mod N, N, aux))`. À la fin, `aux` contient $c \cdot x \bmod N$ et `qs` contient toujours x.
3. Échanger `qs` et `aux` via des `SWAP` qubit à qubit. Maintenant `qs` contient le résultat cx, et `aux` contient l'ancien x.
4. Nettoyer `aux`. Comme `qs` contient maintenant $cx \bmod N$, on veut soustraire $c^{-1} \cdot cx \bmod N = x$ à `aux` pour la ramener à 0. On utilise `InverseModI(c, N)` pour calculer $c^{-1} \bmod N$, puis on applique le même schéma binaire qu'à l'étape 2 mais avec $-c^{-1} \cdot 2^i \bmod N$ comme constante (qu'on obtient avec `modulo - shiftC`).

Pourquoi cette gymnastique avec swap puis inversion ? L'allocation d'une ancilla suivie de `SWAP` est un pattern classique en calcul quantique réversible : on *calcule* le résultat dans un nouveau registre, puis on *échange* l'identité des deux registres. Mais l'ancilla, qui contient maintenant l'ancien x, doit revenir à $|0\rangle$ avant la fin du scope (sinon `is Adj` ne tient pas, et le simulateur lèvera "Qubit released while not in $|0\rangle$ state"). On la décompute en soustrayant $c^{-1} \cdot$ (contenu actuel de qs) mod N, ce qui revient à dérouler le calcul à l'envers.

Pourquoi a-t-on besoin que c et N soient premiers entre eux ? Parce que `InverseModI(c, N)` n'existe que dans ce cas. Si $\gcd(c, N) \neq 1$, c n'a pas d'inverse modulaire et toute la décomputation

casse, ce qui ferait fuiter de l'information sur x dans `aux` et casserait la cohérence quantique. Dans le contexte de Shor, on choisira toujours g avec $\gcd(g, N) = 1$, donc cette condition est garantie.

Indice. La structure est trois boucles consécutives : ajout contrôlé, swap, décomputation. La première et la troisième sont symétriques : même décomposition binaire, même contrôle, mais la troisième utilise `InverseModI(c, N)` comme constante de base et soustrait au lieu d'ajouter (ce qu'on obtient avec `modulo - shiftC` comme constante à ajouter, puisque ajouter $-k \bmod N$ revient à ajouter $N - k$).

Pour le décalage, `(c «< i) % modulo` fonctionne bien tant que `c «< i` ne déborde pas l'entier 64-bit. Dans le cadre du TP avec N à 5 chiffres, vous êtes large.

Test. Quatre tests progressifs à passer avant de continuer.

Test 1 — cas de base.

```
use qs = Qubit[3]; X(qs[1]);
// qs = |010⟩ = 2 (en little-endian)
MultiplyConstantModulo(5, 7, qs);
DumpMachine(); // attendu : |110⟩ = 3 (car 2×5 mod 7 = 3)
```

Test 2 — petit théorème de Fermat. Pour p premier et g premier avec p, $g^{p-1} \equiv 1 \pmod{p}$. Donc multiplier 1 par 3 six fois modulo 7 doit redonner 1.

```
use qs = Qubit[3]; X(qs[0]);
// qs = |100⟩ = 1
for _ in 1..6 { MultiplyConstantModulo(3, 7, qs); }
DumpMachine(); // attendu : |100⟩ = 1
```

Test 3 — inverses modulaires. Si $c \cdot c^{-1} \equiv 1 \pmod{N}$, multiplier par c puis par c^{-1} doit être l'identité. Pour $N = 7$, $5^{-1} = 3$ (car $5 \times 3 = 15 \equiv 1 \pmod{7}$).

```
use qs = Qubit[3]; X(qs[1]);
// qs = |010⟩ = 2
MultiplyConstantModulo(5, 7, qs); // qs devient 10 mod 7 = 3
MultiplyConstantModulo(3, 7, qs); // qs devient 9 mod 7 = 2
DumpMachine(); // attendu : |010⟩ = 2
```

Test 4 — adjoint sur état superposé. C'est le vrai test de la propriété `is Adj`. Une faute de décomputation passe inaperçue sur des états de base, mais éclate sur une superposition.

```
use qs = Qubit[3]; H(qs[0]); H(qs[1]);
// superposition uniforme sur {0, 1, 2, 3}
MultiplyConstantModulo(3, 7, qs);
Adjoint MultiplyConstantModulo(3, 7, qs);
DumpMachine(); // attendu : superposition uniforme initiale sur {0,1,2,3}
// chacun avec amplitude 0.5 et phase 0
```

Si ce dernier test ne retombe pas sur la superposition initiale, votre décomputation est probabilistement correcte mais quantiquement fausse : un résidu d'intrication entre `qs` et l'ancilla reste après le `Adjoint`.

Checkpoint. À ce stade, vous savez :

- transformer une opération non réversible (multiplication) en circuit réversible ;
- utiliser un registre auxiliaire pour préserver l'information ;
- effectuer une décomputation propre via un inverse modulaire.

Vous venez en réalité de construire la porte U_a de Shor. **C'est le cœur arithmétique de l'algorithme.**

Questions (à réfléchir).

— Pourquoi la multiplication n'est-elle pas directement réversible ?
— Que se passerait-il si on ne faisait pas le `SWAP` ?
— Pourquoi l'inverse modulaire est-il nécessaire pour la décomputation ?

2 Partie quantique : QPE itératif

Ces exercices se trouvent dans le fichier `PhaseEstimation.qs`.

La porte U_a que vous venez d'écrire a une propriété qui va nous servir : ses valeurs propres encodent la **période multiplicative** de a modulo N. Plus précisément, si l'on note r l'ordre de a dans $(\mathbb{Z}/N\mathbb{Z})^*$ (le plus petit entier tel que $a^r \equiv 1 \pmod{N}$), alors U_a a des vecteurs propres de valeurs propres $e^{2\pi i s/r}$ pour $s \in \{0, 1, \ldots, r-1\}$. Récupérer ces phases, c'est récupérer la période r. C'est exactement le rôle de QPE.

Pourquoi une variante itérative ?

Le QPE standard du TP3 utilise n qubits ancillaires pour une précision de n bits. Pour Shor, on a besoin de $2n+1$ bits de précision (justifié par la théorie des fractions continues : voir l'annexe), où n est le nombre de bits de N. Pour $N = 35\,333$, $n = 16$, donc on aurait besoin de 33 ancillas *en plus* des 16 qubits de `qs`. Sur un simulateur, ça commence à piquer.

La variante **itérative** économise drastiquement : un *seul* qubit ancillaire, réutilisé $p = 2n + 1$ fois. À chaque itération, on extrait un bit de la phase et on en accumule l'information dans une variable classique. C'est moins parallélisable qu'un QPE standard mais infiniment plus économe en qubits.

Vue d'ensemble. Jusqu'ici, vous avez construit la transformation U_a.

Objectif de cette section :

extraire la période r en mesurant une phase quantique.

Pipeline :

$$U_a \to \text{QPE} \to \frac{s}{r} \to \text{fractions continues} \to r$$

Intuition (essentielle). Le QPE itératif peut être vu comme une reconstruction binaire de la phase.

On cherche à écrire :

$$\varphi = 0.b_1 b_2 b_3 \ldots b_p$$

À chaque itération :

— on mesure un bit b_i ;
— on corrige la phase pour retirer l'influence des bits déjà connus ;
— on passe au bit suivant.

Autrement dit :

*on reconstruit la phase **bit par bit**, du plus significatif au moins significatif.*

2.1 Implémentation de `EstimateFrequency`

La signature :

```
operation EstimateFrequency(generator: Int, modulo: Int, moduloBitSize: Int) :
Int
```

Les variables d'entrée, en détail.

- `generator: Int` est le générateur g dont on cherche l'ordre r modulo N. Précondition : $1 < g < N$ et $\gcd(g, N) = 1$. La coprimalité est imposée parce que sans elle, g n'a pas d'ordre fini dans $(\mathbb{Z}/N\mathbb{Z})^*$.
- `modulo: Int` est le module N. C'est l'entier qu'on cherche à factoriser.
- `moduloBitSize: Int` est le nombre de bits nécessaires pour représenter N, autrement dit $\lceil \log_2 N \rceil$. Pour $N = 15$, c'est 4 ; pour $N = 35\,333$, c'est 16. Vous obtenez cette valeur avec `BitSizeI(N)` en amont (dans `EstimatePeriod`).
- **Sortie** : un entier `frequency` dans $[0, 2^{2n+1})$ tel que `frequency` / $2^{2n+1} \approx s/r$ pour un s aléatoire uniformément tiré dans $\{0, 1, \ldots, r-1\}$. Le post-traitement classique se chargera de remonter à r.

Le squelette algorithmique :

1. Initialiser une variable classique `frequency = 0` et calculer `bitsPrecision = 2 * moduloBitSize + 1`.
2. Allouer un registre `qs` de `moduloBitSize` qubits, le préparer à $|1\rangle$ (avec `X(qs[0])`, puisqu'on est en little-endian).
3. Allouer un qubit ancillaire `c`.
4. Pour i allant de `bitsPrecision - 1` à `0` (sens décroissant) :
   - Appliquer $H(c)$.
   - Appliquer `Controlled ApplyUGate` : $U_a^{2^i}$ contrôlé par c, agissant sur `qs`.
   - Appliquer `R1Frac(frequency, bitsPrecision - 1 - i, c)` pour corriger la phase en fonction des bits déjà mesurés.
   - Appliquer $H(c)$ à nouveau.
   - Mesurer c. Si le résultat est `One`, ajouter $2^{bitsPrecision-1-i}$ à `frequency`, et appliquer $X(c)$ pour le remettre à $|0\rangle$.
5. Réinitialiser le registre `qs` et retourner `frequency`.

Pourquoi préparer `qs` à $|1\rangle$ et pas à $|0\rangle$? Parce que $|1\rangle$ n'est pas un vecteur propre exact de U_a, mais c'est une superposition uniforme des vrais vecteurs propres $|u_s\rangle = \frac{1}{\sqrt{r}} \sum_k e^{-2\pi i s k/r} \left|a^k \bmod N\right\rangle$. La mesure projette donc sur l'un des $|u_s\rangle$ au hasard, et on récupère s/r pour un s uniforme dans $\{0, \ldots, r-1\}$. C'est exactement ce qu'il faut pour appliquer ensuite l'algorithme des fractions continues.

À quoi sert `R1Frac` ? En QPE itératif, après avoir mesuré les bits de poids fort de la phase, il faut les "soustraire" de l'ancilla avant la prochaine itération pour ne pas les recompter. `R1Frac(num, denom, q)` applique une rotation $e^{i\pi \cdot \mathrm{num}/2^{\mathrm{denom}}}$ à $|1\rangle$ du qubit `q`, ce qui réalise exactement cette correction de phase. Sans `R1Frac`, les bits déjà mesurés viennent perturber les mesures suivantes.

Pourquoi $X(c)$ et pas `Reset(c)` après la mesure ? `Reset(c)` ferait le travail fonctionnellement, mais il intercepte le résultat de la mesure par un appel interne à M et bascule le qubit avec un X si nécessaire. C'est exactement ce qu'on fait à la main : on connaît déjà le résultat de la mesure (on vient de l'utiliser pour décider si on incrémente `frequency`), donc autant appliquer le X directement sans repasser par la mesure. Plus propre, plus rapide, et conceptuellement aligné avec la sémantique de QPE itératif.

Checkpoint. Vous avez maintenant implémenté :

— une version itérative de QPE ;
— l'extraction d'une phase quantique sous forme classique ;
— une approximation de la fraction s/r.

À ce stade, toute la partie quantique est terminée. Le reste de l'algorithme est purement classique.

2.2 L'optimisation `ExpModI`

La version naïve appliquerait `Controlled ApplyUGate` (avec `power = 1`) exactement 2^i fois pour réaliser $U_a^{2^i}$. Pour $i = 32$, c'est 2^{32} appels, soit autant qu'il y a de secondes dans 136 ans. Sur un simulateur. Bonne chance.

L'optimisation centrale est la suivante : comme U_a est une multiplication par a modulo N, on a

$$U_a^{2^i} |x\rangle = \left| a^{2^i} \cdot x \bmod N \right\rangle = U_{a^{2^i} \bmod N} |x\rangle$$

Donc au lieu d'appliquer U_a exactement 2^i fois, on calcule **classiquement** la valeur de $a^{2^i} \bmod N$ avec `ExpModI(a, 2^i, N)` (en temps logarithmique grâce à l'exponentiation rapide) et on applique *une seule fois* la porte $U_{a^{2^i} \bmod N}$. Le cœur de l'avantage quantique se cache ici : on traite des nombres énormes en utilisant uniquement leur valeur modulo N, qu'on peut tenir dans nos n qubits.

C'est précisément ce que fait `ApplyUGate` :

```
operation ApplyUGate(generator: Int, modulo: Int, power: Int, qs: Qubit[]) : Unit
is Adj + Ctl {
    MultiplyConstantModulo(ExpModI(generator, power, modulo), modulo, qs)
}
```

Indice. La structure de la boucle est presque toujours mal vue du premier coup à cause de `R1Frac`. Le contrat est : *après la mesure du bit i, le résultat partiel* `frequency` *contient déjà les bits de poids supérieur, et il faut soustraire leur contribution à la phase avant de mesurer le bit suivant.* La position du `R1Frac` entre le contrôle-U et le second Hadamard n'est pas négociable. Si vous le mettez ailleurs, soit vous corrigez la mauvaise phase, soit vous corrigez avec le mauvais signe.

Astuce d'implémentation : le pattern `within { H(c); } apply { ... }` encapsule proprement les deux Hadamards autour du contrôle-U et du `R1Frac`. Cela évite de dupliquer le `H(c)`.

Test. Deux niveaux de validation.
Test 1 — single shot, valeurs admissibles. Pour $N = 15$, $g = 2$, $r = 4$, `bitsPrecision = 9`. Les fréquences possibles sont *exactement* $\{0, 128, 256, 384\}$ (puisque $1/4$ est exactement représentable sur 9 bits). Lancez :

```
let frequency = EstimateFrequency(2, 15, 4);
Message($"Fréquence : {frequency}");
let isValid = frequency == 0 or frequency == 128
  or frequency == 256 or frequency == 384;
Fact(isValid, $"Fréquence {frequency} inattendue");
```

À chaque run, le `Fact` doit passer. Si vous obtenez 100 ou 200 ou n'importe quelle autre valeur, votre `R1Frac` est mal placé ou mal calculé.
Test 2 — distribution statistique. Sur 40 runs, vous devez voir les quatre valeurs avec à peu près la même fréquence (10 chacune en moyenne, mais avec la variance d'une distribution multinomiale).

```
mutable count0 = 0; mutable count128 = 0;
mutable count256 = 0; mutable count384 = 0;
for _ in 1..40 {
  let freq = EstimateFrequency(2, 15, 4);
  if freq == 0 { count0 += 1; }
  elif freq == 128 { count128 += 1; }
  elif freq == 256 { count256 += 1; }
  elif freq == 384 { count384 += 1; }
}
Message($"0={count0}, 128={count128}, 256={count256}, 384={count384}");
```

Un exemple de sortie validée : `0=7, 128=14, 256=12, 384=7`. Pas exactement uniforme, mais clairement les quatre valeurs sont représentées.
Test 3 — modulo non puissance de 2. Pour $N = 21$, $g = 2$, $r = 6$. Comme $1/6$ n'est pas représentable exactement sur 11 bits, les fréquences se répartissent sur 6 "bins" centrés autour de $s \cdot 2^{11}/6$ pour $s = 0, \ldots, 5$. Sur 30 runs, les 6 bins doivent tous être visités. C'est un test plus exigeant qui valide que QPE fonctionne aussi pour des phases irrationnelles.

Questions (à réfléchir).

- Pourquoi prépare-t-on $|1\rangle$ et non $|0\rangle$?
- Que se passe-t-il si on enlève `R1Frac` ?
- Pourquoi mesure-t-on du bit de poids fort au bit de poids faible ?

3 Partie classique : tout sauf la phase

Ces exercices se trouvent dans le fichier `ClassicalPart.qs`. Toute la suite est du post-traitement classique : conversion de la fréquence en période via les fractions continues, extraction des facteurs depuis la période, et orchestration de l'ensemble.

Vue d'ensemble. La partie quantique vous a donné une approximation de $\frac{s}{r}$.

Objectif ici :

- reconstruire r à partir de cette fraction ;
- utiliser r pour factoriser N.

Pipeline final :

$$\frac{s}{r} \to r \to \gcd(g^{r/2} \pm 1, N) \to \text{facteurs}$$

3.1 `PeriodFromFrequency`

La signature attendue :

```
function PeriodFromFrequency(N: Int, frequency: Int, bitsPrecision: Int, currentDivisor: Int) : Int
```

Les variables d'entrée, en détail.

- `N: Int` est l'entier à factoriser. Sert uniquement de borne supérieure pour le dénominateur du convergent renvoyé par les fractions continues.
- `frequency: Int` est la fréquence brute renvoyée par `EstimateFrequency`, à valeurs dans $[0, 2^{2n+1})$.
- `bitsPrecision: Int` est le nombre de bits de précision utilisé par QPE, soit $2n + 1$ où $n = \lceil \log_2 N \rceil$.
- `currentDivisor: Int` permet de combiner la nouvelle période trouvée avec un diviseur déjà connu via le PPCM. Pour un premier appel, on passe 1 (la valeur neutre du PPCM).
- **Sortie** : la période r (ou un diviseur de r si le tirage s a fait perdre de l'information).

Le problème. QPE renvoie une approximation entière `frequency` de $2^p \cdot s/r$, où s est aléatoire dans $\{0, \ldots, r-1\}$ et r est l'ordre recherché. On a donc l'approximation rationnelle

$$\frac{\text{frequency}}{2^p} \approx \frac{s}{r}$$

et il faut récupérer r à partir de cette fraction. La théorie des **fractions continues** dit que si l'approximation est suffisamment précise (précisément, si $|\text{frequency}/2^p - s/r| < 1/(2r^2)$), alors la fraction s/r *en forme réduite* apparaît parmi les convergents de la fraction continue de frequency$/2^p$. Le théorème garantit qu'on peut récupérer r exactement.

C'est la justification du choix $p = 2n + 1$ où $n = \lceil \log_2 N \rceil$: on a alors $2^p \geq 2N^2 > 2r^2$ (puisque $r < N$), ce qui assure la précision suffisante.

L'algorithme à implémenter :

1. Appeler `ContinuedFractionConvergentI((frequency, 2^p), N)` qui renvoie un tuple `(numerator, denominator)` représentant le meilleur convergent rationnel de frequency$/2^p$ dont le dénominateur est $\leq N$. C'est la période candidate.
2. Normaliser les signes : la fonction peut renvoyer des valeurs négatives (selon le sens de la convergence), il faut prendre les valeurs absolues avec `AbsI`.
3. Combiner avec `currentDivisor` via le PPCM :

$$\text{finalPeriod} = \frac{\text{periodAbs} \cdot \text{currentDivisor}}{\gcd(\text{currentDivisor}, \text{periodAbs})}$$

   Cette combinaison permet de raffiner progressivement la période quand on a déjà des estimations partielles. Pour le premier appel, `currentDivisor = 1` et la formule renvoie simplement `periodAbs`.

Pourquoi le PPCM ? Parce que QPE peut renvoyer un *diviseur* de la vraie période plutôt que la période elle-même (si la fraction s/r n'est pas en forme réduite, le dénominateur récupéré sera $r/\gcd(s, r)$, donc un diviseur). En combinant plusieurs essais via PPCM, on reconstruit progressivement la vraie période. Dans le cadre minimaliste de ce TP, on n'exploite pas ce raffinement (un seul appel avec `currentDivisor = 1`), mais l'interface est là pour une extension.

Pourquoi `function` et pas `operation` ? Parce que tout ici est purement classique, déterministe et sans effet de bord. Q# distingue les deux : une `function` est mathématiquement pure (mêmes entrées ⇒ mêmes sorties, pas d'aléa, pas de qubits), alors qu'une `operation` peut tirer de l'aléatoire ou manipuler des qubits. Si vous mettez `operation` ici, ça compilera mais c'est trompeur sur la nature de la fonction.

Indice. La fonction est très courte, essentiellement trois lignes : appel de `ContinuedFractionConvergentI`, normalisation des signes avec `AbsI`, calcul du PPCM. La seule subtilité est de bien désencapsuler le tuple renvoyé par `ContinuedFractionConvergentI` (la syntaxe Q# est `let (num, period) = ...`).

Attention au shadowing : si vous nommez le résultat final `period` alors que `period` était déjà le dénominateur extrait du tuple, Q# l'accepte mais c'est confus à relire. Nommez la valeur finale `finalPeriod` pour la clarté.

Test. Cinq cas déterministes pour $N = 15$, $p = 9$ ($2^p = 512$) :

```
let p1 = PeriodFromFrequency(15, 128, 9, 1);
// attendu : 4 (car 128/512 = 1/4, période 4)
let p2 = PeriodFromFrequency(15, 256, 9, 1);
// attendu : 2 (car 256/512 = 1/2, diviseur de 4)
let p3 = PeriodFromFrequency(15, 384, 9, 1);
// attendu : 4 (car 384/512 = 3/4, période 4)
let p4 = PeriodFromFrequency(15, 0, 9, 1);
// attendu : 0 ou 1 (cas dégénéré ; EstimatePeriod filtre ce cas en
amont)
let p5 = PeriodFromFrequency(15, 128, 9, 2);
// attendu : 4 (lcm(2, 4) = 4 ; combinaison avec un diviseur précédent)
```

Comme la fonction est purement classique, ces sorties doivent être strictement reproductibles : si vous obtenez autre chose, c'est que votre formule de PPCM est erronée ou que vous avez oublié `AbsI`.

3.2 `EstimatePeriod`

La signature :

```
operation EstimatePeriod(g: Int, N: Int) : Int
```

Les variables d'entrée, en détail.

- `g: Int` est le générateur dont on cherche l'ordre. Précondition : $\gcd(g, N) = 1$. Si la condition n'est pas vérifiée, le `Fact` fait échouer le programme avec un message d'erreur explicite.
- `N: Int` est l'entier à factoriser.
- **Sortie** : la période r telle que $g^r \equiv 1 \pmod{N}$, ou 1 en cas d'échec de l'estimation quantique (la fréquence renvoyée par QPE était 0).

Cette opération orchestre la partie quantique et appelle le post-traitement classique. Très peu de code à écrire :

1. Vérifier que $\gcd(g, N) = 1$ avec `Fact`. Si la précondition échoue, l'algorithme est mal utilisé.
2. Calculer `bitSize = BitSizeI(N)`, le nombre de bits nécessaires pour représenter N.
3. Appeler `EstimateFrequency(g, N, bitSize)` pour obtenir la fréquence quantique.
4. Si la fréquence est non nulle, appeler `PeriodFromFrequency` avec `currentDivisor = 1`.
5. Si la fréquence est nulle, retourner 1 (signal d'échec).

Pourquoi retourner 1 en cas d'échec ? Parce que 1 est une période "triviale" (toute valeur a 1 comme diviseur de sa période). L'appelant testera ce cas et relancera avec un nouveau g. C'est le mécanisme de retry qui sauve l'algorithme face à la stochasticité.

Pourquoi pas une exception sur fréquence nulle ? Parce qu'on est dans un algorithme probabiliste. Une fréquence nulle est un événement attendu, pas une faute. On signale l'échec par une valeur de retour spéciale et on laisse l'appelant décider de la stratégie.

Indice. Pas vraiment de piège ici, c'est essentiellement de l'enchaînement d'appels. La seule chose à retenir est l'ordre : `BitSizeI` avant l'appel quantique, et la précision $2 \cdot \text{bitSize} + 1$ passée à `PeriodFromFrequency`.

Test. Trois cas canoniques à lancer plusieurs fois pour observer la variance :

- `EstimatePeriod(2, 15)` doit retourner 4 (la bonne période), 2 (un diviseur, si la mesure tombe sur $s = 2$), ou 1 (échec, si fréquence = 0).
- `EstimatePeriod(7, 15)` : idem, l'ordre de 7 modulo 15 est aussi 4.
- `EstimatePeriod(2, 21)` doit retourner 6 ou un diviseur (2 ou 3).

Lancez chaque test 10 fois pour observer la distribution des résultats. La bonne période apparaît typiquement dans environ 50 % des cas ; le reste se répartit entre les diviseurs et l'échec à 1.

3.3 `MaybeFactorsFromPeriod`

La signature :

```
function MaybeFactorsFromPeriod(N: Int, g: Int, r: Int) : (Bool, (Int, Int))
```

Les variables d'entrée, en détail.

- `N: Int` est l'entier à factoriser.
- `g: Int` est le générateur utilisé pour trouver la période.
- `r: Int` est la période trouvée. Si elle est invalide (impaire ou conduisant à un facteur trivial), la fonction renvoie un signal d'échec.
- **Sortie** : un tuple `(Bool, (Int, Int))`. Le booléen indique si la factorisation a réussi. Les deux entiers sont les facteurs, ou `(1, 1)` en cas d'échec.

Une fois la période r obtenue, on en extrait les facteurs via une identité algébrique simple :

$$g^r \equiv 1 \pmod{N} \quad \Longrightarrow \quad (g^{r/2} - 1)(g^{r/2} + 1) \equiv 0 \pmod{N}$$

Si r est pair, $g^{r/2}$ est défini. Si en plus $g^{r/2} \not\equiv -1 \pmod{N}$, alors ni $g^{r/2} - 1$ ni $g^{r/2} + 1$ ne sont des multiples de N, ce qui veut dire que les deux contiennent chacun un facteur non trivial de N. On les extrait avec `GreatestCommonDivisorI`.

L'algorithme :

1. Si r est impair, retourner `(false, (1, 1))`. Il faudra réessayer avec un autre g.
2. Calculer $h = g^{r/2} \bmod N$ avec `ExpModI(g, r / 2, N)`.
3. Si $h \equiv -1 \pmod{N}$ (autrement dit $h = N - 1$), retourner `(false, (1, 1))`. C'est le cas trivial.
4. Sinon, calculer $f_1 = \gcd(h - 1, N)$ et $f_2 = \gcd(h + 1, N)$, prendre $f = \max(f_1, f_2)$, et vérifier que $1 < f < N$.
5. Si oui, retourner `(true, (f, N / f))`. Sinon, `(false, (1, 1))`.

Pourquoi r doit être pair ? Parce qu'on a besoin de $g^{r/2}$ entier. Si r est impair, on est coincé. La probabilité que r soit pair pour un g aléatoire est supérieure à $1/2$ (résultat non trivial de théorie des nombres), donc en tirant plusieurs g, on finit par tomber sur une période paire.

Pourquoi exclure $g^{r/2} \equiv -1 \pmod{N}$? Parce qu'alors $g^{r/2} - 1 \equiv -2 \pmod{N}$ et $g^{r/2} + 1 \equiv 0 \pmod{N}$, donc $\gcd(g^{r/2} + 1, N) = N$, un facteur trivial. On n'apprend rien sur N. Comme pour la parité, ce cas survient avec probabilité $\leq 1/2$, donc en réessayant, on finit par tomber sur un g utilisable.

Pourquoi le max *entre f_1 et f_2 ?* Parce que l'un des deux peut être 1 ou N même quand l'autre est non trivial. Prendre le maximum maximise la chance d'obtenir un facteur exploitable. À

titre d'exemple, pour $N = 15$, $g = 2$, $r = 4$: $h = 4$, $\gcd(3, 15) = 3$, $\gcd(5, 15) = 5$. Les deux sont non triviaux et exploitables, mais l'algorithme prendra 5 (le max), donnant la factorisation $15 = 5 \times 3$.

Indice. La fonction est purement classique (`function`, pas `operation`). Cinq lignes de logique, deux `return` de bonne fin et deux de mauvaise fin. La seule subtilité est de bien comparer h à $N - 1$ et non à -1 (Q# n'a pas d'arithmétique modulaire native sur les Int signés).

Test. Six cas déterministes (la fonction est purement classique) à passer :

```
let (ok1, (a1, b1)) = MaybeFactorsFromPeriod(15, 2, 4);
// attendu : (true, (5, 3)) car 2^2 mod 15 = 4
let (ok2, (a2, b2)) = MaybeFactorsFromPeriod(15, 4, 2);
// attendu : (true, (5, 3)) car 4^1 mod 15 = 4
let (ok3, (a3, b3)) = MaybeFactorsFromPeriod(21, 2, 6);
// attendu : (true, (7, 3)) car 2^3 mod 21 = 8
let (ok4, _) = MaybeFactorsFromPeriod(15, 7, 4);
// attendu : (true, ...) car 7^2 mod 15 = 49 mod 15 = 4
let (ok5, _) = MaybeFactorsFromPeriod(15, 2, 3);
// attendu : (false, (1, 1)) car r impair
let (ok6, _) = MaybeFactorsFromPeriod(15, 14, 2);
// attendu : (false, (1, 1)) car 14^1 mod 15 = 14 = N-1 (cas trivial)
```

Si l'un de ces six tests ne donne pas le résultat attendu, le bug est dans la condition exacte que vous testez sur `halfPower`.

3.4 `FactorSemiprimeInteger`

La signature :

```
operation FactorSemiprimeInteger(n: Int) : (Int, Int)
```

Les variables d'entrée, en détail.

- `n: Int` est l'entier à factoriser. On suppose que c'est un semipremier : $n = p \cdot q$ avec p, q deux nombres premiers. C'est l'hypothèse minimale pour que Shor donne une factorisation simple (l'algorithme se généralise mais devient nettement plus subtil sur des entiers avec plus de deux facteurs).
- **Sortie** : un couple (a, b) tel que $a \cdot b = n$ et $1 < a, b < n$. L'ordre n'est pas garanti (vous pouvez avoir $(3, 5)$ ou $(5, 3)$ pour $n = 15$).

C'est la fonction-chapeau qui orchestre tout. Elle gère les cas faciles, la boucle de retry, et le tirage aléatoire de g.

1. Si n est pair, retourner $(n/2, 2)$ immédiatement. Pas la peine d'allumer le quantique pour ça.
2. Initialiser `foundFactors = false`, `factors = (1, 1)`, `attempt = 1`.
3. Boucle `repeat ... until foundFactors fixup { ... }` :
   - Tirer un g aléatoire dans $[1, n - 1]$ avec `DrawRandomInt`.
   - Si $\gcd(g, n) = 1$, lancer la machinerie quantique : `EstimatePeriod` puis `MaybeFactorsFromPeriod`.
   - Sinon (cas chanceux), $\gcd(g, n)$ est déjà un facteur non trivial : on l'utilise directement, sans quantique.
   - Incrémenter `attempt`, et faire `fail` si `attempt > 100`.
   - Le bloc `fixup` affiche un message et la boucle reprend.
4. Retourner `factors`.

Pourquoi la borne de 100 essais ? Sécurité. En théorie, chaque tentative réussit avec probabilité $\geq 1/2$, donc la probabilité d'échouer 100 fois est $\leq 2^{-100}$. C'est plutôt par sécurité contre un bug d'implémentation qu'on met cette borne ; si elle saute, vous avez un vrai problème ailleurs.

Pourquoi tester d'abord n pair ? Parce que c'est immédiat et que Shor avec un g aléatoire dans $[1, n-1]$ marche très mal sur les multiples de 2 (l'algorithme suppose n semipremier impair). C'est une optimisation propre.

Pourquoi le cas $\gcd(g, n) \neq 1$ *?* Pure chance. Si le tirage aléatoire tombe sur un g qui partage un facteur avec n, on a gagné sans faire de calcul quantique. Ça arrive avec probabilité non négligeable pour des n avec peu de facteurs ; pour $n = 21$ par exemple, 8 valeurs de $g \in [1, 20]$ sont non coprimes avec 21 (les multiples de 3 ou de 7), soit 40 % de chance d'éviter le quantique en un tirage. C'est gratuit, donc autant en profiter.

Indice. La structure `repeat ... until ... fixup { ... }` est spécifique à Q#. Le `fixup` s'exécute après chaque échec (chaque tour où la condition `until` reste fausse), et c'est l'endroit idéal pour un `Message` qui indique qu'on retente. La condition `until foundFactors` se teste après chaque exécution du corps de boucle, pas du fixup.

La déclaration `mutable` est obligatoire pour les variables modifiées dans la boucle ; les autres restent immuables.

Test. Cas de validation, du plus simple au plus ambitieux.
Test 1 — petits semipremiers, attendus en quelques secondes :

```
let (a, b) = FactorSemiprimeInteger(15);
// attendu : (5, 3) ou (3, 5)
let (a, b) = FactorSemiprimeInteger(21);
// attendu : (3, 7) ou (7, 3) ou (3, 7) par chemin classique
let (a, b) = FactorSemiprimeInteger(35);
// attendu : (5, 7) ou (7, 5)
```

Test 2 — semipremier non trivial, attendu en quelques minutes selon les tentatives nécessaires :

```
let (a, b) = FactorSemiprimeInteger(35333);
// attendu : (397, 89) ou (89, 397)
```

Note importante sur $n = 21$. Comme indiqué plus haut, environ 40 % des tirages de g pour $n = 21$ tombent sur un non coprime de 21, ce qui déclenche le chemin classique chanceux sans exécuter QPE. Si vous voulez *forcer* la démonstration du chemin quantique, lancez la factorisation plusieurs fois et observez les messages : tant que vous voyez `"We have guessed a divisor by accident"`, le quantique n'a pas tourné. $n = 15$ est plus fiable pour démontrer le pipeline quantique complet, parce que seules 6 valeurs sur 14 sont non coprimes avec 15.

Avertissement sur les temps de calcul : pour $n = 35\,333$, l'algorithme demande 16 qubits de données plus l'ancilla, et 33 itérations de QPE itératif. Sur un simulateur, comptez plusieurs minutes par tentative ; si la première rate, ça peut prendre une dizaine de minutes au total. Pour observer la variance, restez sur les petits cas ($n \leq 35$).

A Pourquoi tout cela fonctionne (la version friendly)

L'algorithme de Shor a la réputation d'être impénétrable. C'est largement injuste : la partie quantique est essentiellement une réutilisation de QPE, et la magie est dans une chaîne de réductions classiques qui transforment "factoriser N" en "trouver une période d'une fonction". Cette annexe fait le pont, étape par étape, avec aussi peu de prérequis que possible.

Étape 1 : factoriser un entier équivaut à trouver un ordre

Soit $N = pq$ un produit de deux nombres premiers impairs (le cas semipremier). On veut récupérer p et q.

Choisissons un entier g pris au hasard dans $[1, N-1]$ avec $\gcd(g, N) = 1$ (sinon on a déjà gagné, `gcd` est un facteur). Dans le groupe $(\mathbb{Z}/N\mathbb{Z})^*$, g a un **ordre** r, défini comme le plus petit entier strictement positif tel que $g^r \equiv 1 \pmod N$. C'est la *période* de la suite $g^0, g^1, g^2, \ldots$ vue modulo N.

Maintenant, voici l'identité magique. Si r est pair, alors :

$$g^r - 1 \equiv 0 \pmod N \quad \Longleftrightarrow \quad (g^{r/2} - 1)(g^{r/2} + 1) \equiv 0 \pmod N$$

Donc N divise le produit $(g^{r/2} - 1)(g^{r/2} + 1)$. Si en plus $g^{r/2} \not\equiv \pm 1 \pmod N$, alors N ne divise ni $g^{r/2} - 1$ ni $g^{r/2} + 1$ tout seul, mais il partage forcément un facteur premier avec chacun des deux. Et ce facteur, on le récupère avec un simple $\gcd(g^{r/2} \pm 1, N)$.

Donc : **si l'on sait calculer l'ordre r, on sait factoriser N.**

Mais r peut être impair, ou $g^{r/2}$ peut être -1. Vrai. Heureusement, un théorème de théorie des nombres (Miller-Rabin, démonstration élémentaire) garantit qu'au moins $1/2$ des g pris au hasard donnent un r pair avec $g^{r/2} \not\equiv \pm 1$. Donc en tirant plusieurs g, on tombe rapidement sur un bon candidat. Voilà pourquoi l'algorithme de Shor est *probabiliste* : il peut rater une tentative, mais chaque échec coûte un essai supplémentaire et la probabilité de succès cumulée tend vite vers 1.

Le problème "factoriser N" s'est réduit à "calculer r, l'ordre de g modulo N". Classiquement, calculer r demande de tester $g^1, g^2, \ldots, g^r$ jusqu'à retomber sur 1, ce qui prend $O(r) = O(N)$ opérations dans le pire cas. C'est exactement aussi cher que factoriser à la main, donc on n'a rien gagné. La beauté de Shor est qu'on peut calculer r en quantique *exponentiellement plus vite.*

Étape 2 : QPE révèle la période

La clé est la suivante : la porte $U_a : |x\rangle \to |ax \bmod N\rangle$ a une structure spectrale qui encode r. Concrètement, ses vecteurs propres sont

$$|u_s\rangle = \frac{1}{\sqrt{r}} \sum_{k=0}^{r-1} e^{-2\pi i s k/r} \left| a^k \bmod N \right\rangle, \qquad s = 0, 1, \ldots, r-1$$

et leurs valeurs propres sont $e^{2\pi i s/r}$. Si l'on connaissait un $|u_s\rangle$, QPE nous donnerait s/r avec $2n+1$ bits de précision, donc on remonterait à r par les fractions continues. Mais préparer $|u_s\rangle$ demande déjà de connaître r, donc c'est circulaire.

L'astuce : on n'a pas besoin de préparer $|u_s\rangle$, parce que l'état $|1\rangle$ (le qubit de poids faible à 1, le reste à 0) se décompose comme une superposition uniforme des $|u_s\rangle$:

$$|1\rangle = \frac{1}{\sqrt{r}} \sum_{s=0}^{r-1} |u_s\rangle$$

(joli résultat de transformée de Fourier discrète sur le groupe cyclique). Donc en lançant QPE avec $|1\rangle$ comme état initial, la mesure projette aléatoirement sur l'un des $|u_s\rangle$, et l'on récupère s/r pour un s uniforme dans $\{0, \ldots, r-1\}$. C'est exactement ce dont les fractions continues ont besoin.

Étape 3 : les fractions continues récupèrent r à partir de s/r

QPE nous donne une approximation frequency$/2^p$ de s/r. Comment retrouver s et r séparément ? Réponse : algorithme des fractions continues, qui produit tous les meilleurs convergents rationnels a/b d'un nombre réel, par ordre croissant de b.

Le théorème central (Lagrange, XVIIIe siècle) dit : si $|x - p/q| < 1/(2q^2)$ avec $\gcd(p, q) = 1$, alors p/q est l'un des convergents de la fraction continue de x. Donc si frequency$/2^p$ est suffisamment proche de s/r (sous la barre des $1/(2r^2)$), alors s/r *en forme réduite* apparaîtra parmi les convergents, et la fonction `ContinuedFractionConvergentI` de Q# nous le sort.

Pour assurer la précision, il faut $2^p > 2r^2$, et comme $r < N$, on choisit $2^p > 2N^2$, donc $p \geq 2\lceil \log_2 N \rceil + 1$. C'est l'origine du $2n + 1$ qui revient partout.

Petite nuance. La fraction continue récupère s/r *en forme réduite*. Si $\gcd(s, r) = d > 1$, on récupère s'/r' avec $r' = r/d$, un *diviseur* de la vraie période. C'est pour ça que `PeriodFromFrequency` prend un paramètre `currentDivisor` : en combinant plusieurs essais via PPCM, on raffine jusqu'à reconstruire r. Dans le cadre minimal du TP, on n'exploite pas cette fonctionnalité (un seul appel par tentative), mais c'est là pour qui veut faire mieux.

Étape 4 : la complexité, et pourquoi Shor change la donne

Mettons les choses bout à bout :

- Préparation et application des portes U : $O(n^3)$ portes quantiques où $n = \log_2 N$ (chaque addition modulaire est $O(n)$, chaque multiplication est $O(n^2)$, et on en fait $O(n)$ à différentes puissances).
- QPE itératif : $2n+1$ itérations, chacune avec une porte $U_a^{2^i}$ *une seule fois* grâce à `ExpModI`. Total : $O(n^3)$.
- Post-traitement classique : $O(n^3)$ pour les fractions continues et $O(n^3)$ pour le `ExpModI` final, négligeable.

Total : $O(n^3)$ portes pour factoriser un entier de n bits. C'est **polynomial** en n.

À comparer avec le crible algébrique général (GNFS, le meilleur algorithme classique de factorisation), qui tourne en $\exp\left(c\, n^{1/3} (\log n)^{2/3}\right)$, c'est-à-dire **sous-exponentiel** mais quand même très méchant. Pour $n = 2048$ (RSA-2048), GNFS demanderait approximativement 10^{30} opérations ; Shor demanderait 10^{10} portes quantiques. La différence est de vingt ordres de grandeur, et c'est ça que la NSA et le NIST regardent passer chaque matin en buvant leur café.

En une phrase

> *Shor réduit la factorisation à la recherche d'une période modulaire, QPE révèle cette période sous forme d'une fraction s/r, et les fractions continues séparent classiquement s et r. Le seul ingrédient irremplaçable du quantique est l'extraction de la phase ; tout le reste est de l'arithmétique millénaire.*

Si vous retenez cette phrase, vous savez l'essentiel de pourquoi Shor existe et comment il marche. Le reste est du code, des optimisations, et la beauté tragique de la cryptographie post-quantique.

B Réponses aux questions de réflexion

AddConstantModulo

Que se passerait-il si on supprimait le qubit `carry` ?

Sans le qubit `carry`, il devient impossible de distinguer les deux cas :

— $x + c < N$ (pas de débordement),
— $x + c \geq N$ (débordement).

L'information de débordement est essentielle pour décider si l'on doit réajouter N après la soustraction. Sans cette information, le résultat ne correspond plus à une addition modulaire correcte.

De plus, l'opération deviendrait ambiguë et non réversible, car deux entrées différentes pourraient produire la même sortie.

Pourquoi un `Reset` casserait-il la propriété `Adj` ?

`Reset` effectue implicitement une mesure du qubit suivie d'une éventuelle correction. La mesure est une opération non unitaire et irréversible.

Or, une opération admet un adjoint uniquement si elle est unitaire. Utiliser `Reset` empêche donc de construire l'opérateur inverse, ce qui rend la porte incompatible avec `Adj` et casse toute utilisation dans QPE.

MultiplyConstantModulo

Pourquoi la multiplication n'est-elle pas directement réversible ?

La transformation $|x\rangle \mapsto |cx \bmod N\rangle$ écrase l'information sur x. Plusieurs valeurs différentes de x peuvent donner le même résultat modulo N.

Une opération réversible doit être injective (et donc bijective sur l'espace considéré). Cette multiplication seule ne l'est pas, ce qui interdit son implémentation directe.

Que se passerait-il si on ne faisait pas le `SWAP` ?

Sans le `SWAP`, le résultat $cx \bmod N$ resterait dans le registre auxiliaire `aux`, tandis que `qs` contiendrait toujours x.

Cela poserait deux problèmes :

— la fonction ne correspondrait pas à la signature attendue (résultat dans `qs`) ;
— la décomputation ne pourrait pas être effectuée correctement, car elle repose sur le fait que `qs` contient le résultat final.

Pourquoi l'inverse modulaire est-il nécessaire pour la décomputation ?

Après le `SWAP`, le registre auxiliaire contient encore l'ancienne valeur x. Pour le ramener à $|0\rangle$, il faut inverser l'opération qui a été appliquée.

Comme on a calculé $cx \bmod N$, il faut appliquer l'opération inverse, qui correspond à multiplier par $c^{-1} \bmod N$.

Sans cet inverse, il serait impossible d'annuler proprement l'effet de la première étape, et l'ancilla resterait intriquée avec le registre principal.

EstimateFrequency (QPE)

Pourquoi prépare-t-on $|1\rangle$ et non $|0\rangle$?

L'état $|0\rangle$ est un état propre trivial de U_a avec valeur propre 1, ce qui donne une phase nulle et donc aucune information utile.

En revanche, $|1\rangle$ se décompose en une superposition des états propres de U_a, ce qui permet à la mesure de projeter aléatoirement sur un état propre et de révéler une phase non triviale.

Que se passe-t-il si on enlève `R1Frac` ?

Sans `R1Frac`, les bits déjà mesurés de la phase continuent d'influencer les itérations suivantes.

Résultat :

- les bits mesurés ne correspondent plus à la vraie expansion binaire de la phase ;
- la reconstruction de $\frac{s}{r}$ devient incorrecte ;
- QPE échoue systématiquement (valeurs incohérentes).

Pourquoi mesure-t-on du bit de poids fort au bit de poids faible ?

Le QPE itératif reconstruit la phase bit par bit en commençant par les bits les plus significatifs.

Chaque bit mesuré est ensuite utilisé pour corriger la phase avant la mesure suivante. Si l'on procédait dans l'autre sens, les corrections ne pourraient pas être appliquées correctement, car les bits de poids fort (les plus importants) ne seraient pas encore connus.